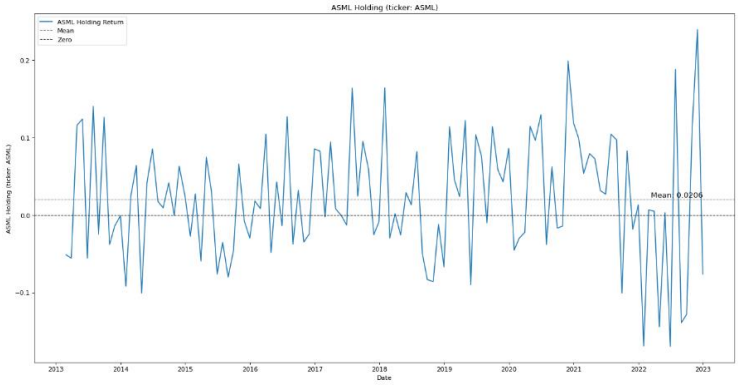
Group 7

Aleksandr Makushin (230015463), Nhat Minh Nguyen (230014328), Jonathan Hou (230036958), Skylar Nguyen (220017332), Jefferson Lays (230014424), Thang Nguyen (230017544)

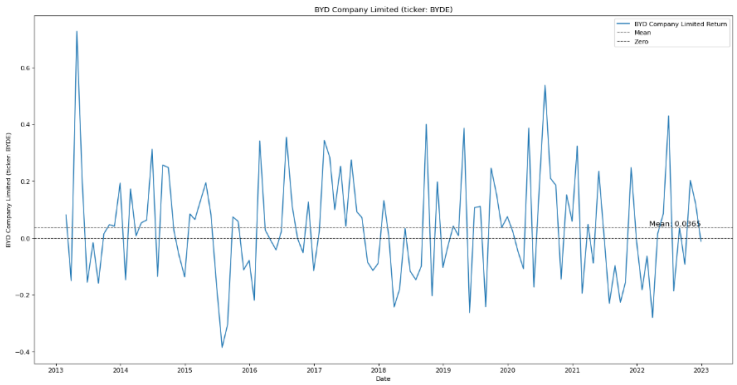
Question 1

Our group selected the following stocks for our portfolio with tickers: ASML, BYDE, LNG, COST, MS, and O.

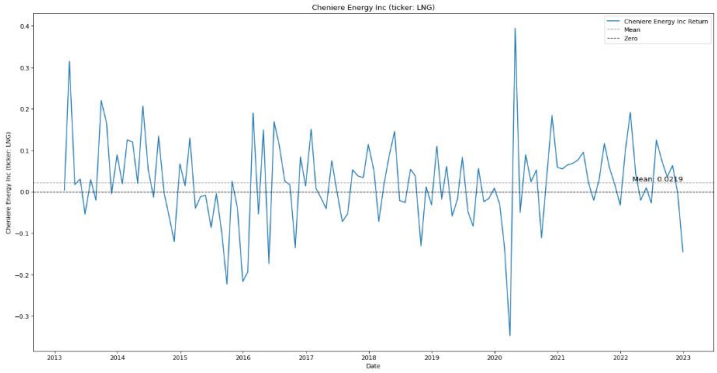
ASML Holding (ticker: ASML) is a semiconductor company headquartered in Veldhoven, The Netherlands. It currently holds a monopoly on extreme ultraviolet lithography, which is essential for producing smaller, faster, and more efficient microchips.



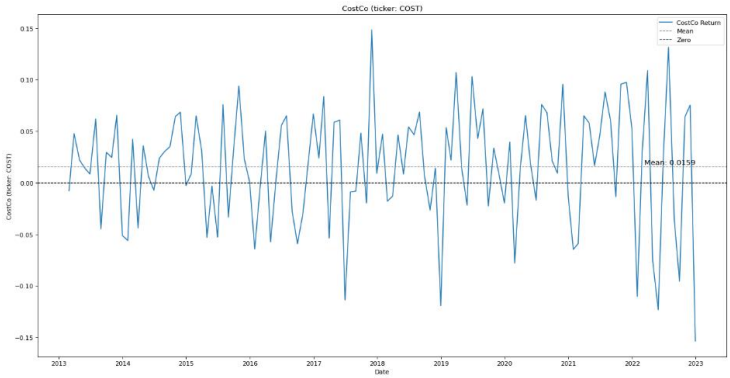
BYD Company Limited (ticker: BYDE) is a Chinese EV company headquartered in Shenzhen, China. Overall, it is currently the world's EV market leader due to sales in China and developing countries, but it is in the process of being a market leader in more developed countries in the west.



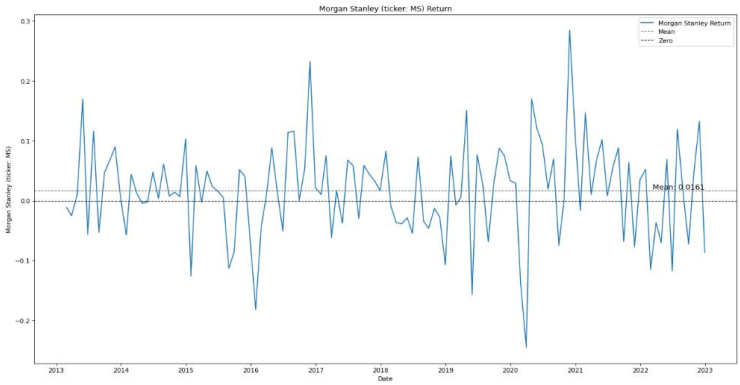
Cheniere Energy Inc. (ticker: LNG) is an international energy company headquartered in Houston, Texas, and is the leading producer and exporter of liquefied natural gas (LNG). Globally, it is the second-largest LNG producer as of 2024.



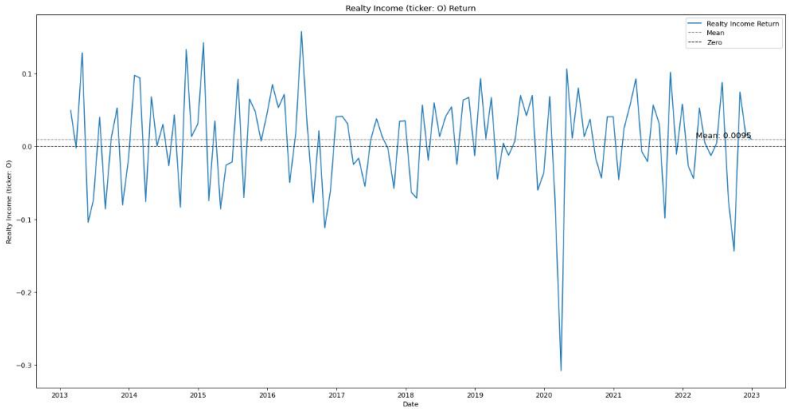
Costco (ticker: COST) is one of the largest membership-only retailers in the world operating in the retail and wholesale industry. It is known for operating on a low margin, high volume business model, passing savings on to its members. Additionally, it distinguishes itself from other retailers through a unique shopping experience through its bulk-selling strategy and limited selection of high-quality items.



Morgan Stanley (ticker: MS) is a global financial service firm headquartered in New York City, USA. The firm operates through three primary business segments: Institutional Securities, Wealth Management, and Investment Management.



Realty Income (ticker: O) operates in the real estate industry, specifically as a real estate investment trust (REIT). It focuses on acquiring and managing commercial real estate properties under long-term net lease agreements.



We selected these six stocks based on the industry in which they operate and the main country of operation. This, we hoped, would allow us to achieve diversification in our portfolio and so reduce the idiosyncratic risk (or so we hoped) as we worried about some of the geopolitical risks that were to come. So, as a result, we invested in Realty Income (O), Cheniere Energy Inc (LNG), Morgan Stanley (MS), Costco (COST), BYD Company Limited (BYDE), ASML Holding (ASML) with weights of 0.23, 0.17, 0.15, 0.10, 0.20, and 0.15 respectively. For example, we invested in LNG as we predicted that the Trump administration will expand domestic oil

and gas production, and so increase revenue of the company. We also invested heavily in Costco and Realty Income as we believed that these companies in which industries operate are recession-proof. After running the code, our portfolio gave us a monthly expected return of 2.030%, standard deviation of 0.392% and a Sharpe ratio of 0.308.

After two months of investing, our portfolio fell by around 10%, meaning we had an unrealized loss of around $10,000. While all our stocks suffered losses, the majority of which were caused by Morgan Stanley (-21%), ASML (-18%), and BYDE (-12%). This was largely due to the global uncertainty and recession fears caused by the tariffs imposed by US President Trump.

From this experience, we clearly saw the impact of the US trade policies on the global financial markets and the need to remain patient and emotionally detached from our stocks. After only a week of losses, our group wanted to close our positions since they were emotionally painful; but when we saw gains in our portfolio, even if they were minor relative to our losses, we were overjoyed. We also learned a major flaw in constructing a portfolio composed entirely of equities, which is that when we sought to hedge our portfolio by investing in an upward-trending commodity (in our case, gold), we had to choose a company that mined it. However, due to the poor financial and short-term outlooks of the companies available for us to invest in, we ultimately decided against it. Whereas, in other cases, we could have hedged our positions through a gold ETF or gold futures, thus mitigating the losses we would have experienced on a stock such as Morgan Stanley (which has fallen around 21% to date).

In hindsight, before investing, we should have constructed an efficient frontier and then select a portfolio that would maximize the Sharpe ratio.

Question 2

	Mean Return	Standard Deviation	Sharpe Ratio
ASML	0.020561	0.078410	0.249474
BYDE	0.036530	0.189052	0.187938
COST	0.015897	0.056265	0.264773
LNG	0.021899	0.102519	0.203859
MS	0.016125	0.080420	0.188079
O	0.009460	0.066433	0.127354

As shown by the data above, BYDE has the highest monthly mean return of about 3.653%, followed by LNG at around 2.189%. Meanwhile, O has the lowest monthly mean return of around 0.946%. With respect to the volatility, BYDE is the riskiest stock in our portfolio, given a monthly standard deviation of around 18.905% which indicates a higher risk. This is followed by LNG which has a monthly standard deviation of around 10.252%. Whereas COST has a monthly standard deviation of around 5.627%, implying stability and lower risk. Also, COST has the highest Sharpe ratio at around 26.477%, which suggests that this stock, within the portfolio, is better at compensating investors for each unit of risk taken. However, O has the lowest Sharpe ratio out of all the stocks in the portfolio at around 3.653%, which implies that it has the worst risk-adjusted performance.

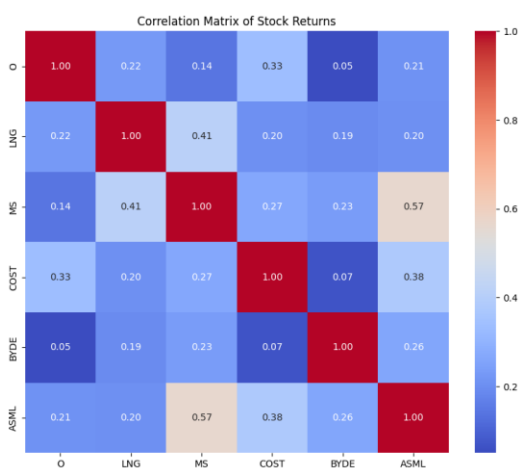
Question 3

Covariance Matrix of Stock Returns:

	ASML	BYDE	COST	LNG	MS	O
ASML	0.006	0.004	0.002	0.002	0.004	0.001
BYDE	0.004	0.036	0.001	0.004	0.004	0.001
COST	0.002	0.001	0.003	0.001	0.001	0.001
LNG	0.002	0.004	0.001	0.011	0.003	0.002
MS	0.004	0.004	0.001	0.003	0.006	0.001
O	0.001	0.001	0.001	0.002	0.001	0.004

Correlation Matrix of Stock Returns:

	ASML	BYDE	COST	LNG	MS	O
ASML	1.000	0.261	0.379	0.205	0.566	0.209
BYDE	0.261	1.000	0.069	0.192	0.234	0.050
COST	0.379	0.069	1.000	0.204	0.265	0.334
LNG	0.205	0.192	0.204	1.000	0.414	0.225
MS	0.566	0.234	0.265	0.414	1.000	0.142
O	0.209	0.050	0.334	0.225	0.142	1.000



As demonstrated by the correlation matrix of stock returns, Morgan Stanley and ASML Holding have the highest positive correlation at 0.566, suggesting these two stocks move relatively closely together. This could reflect similar market influences. Costco and ASML also show a moderately high correlation of 0.379, indicating some level of co-movement. Furthermore, Realty Income and Costco

have a correlation of 0.334, which might reflect some broader market dynamics but still indicates only a mild relationship. Consequently, the low correlations among our selected stocks suggest that the portfolio is well-diversified, effectively minimizing idiosyncratic risk.

Whereas BYDE is generally the least correlated with the other stocks, which could be a reflection of different market exposure. It has a low positive correlation with Realty Income of 0.05 and Costco of 0.07, implying these two stocks are nearly uncorrelated.

Question 4

To identify an optimal long only portfolios of stocks and construct the efficient frontier, we began by defining a finite set of weight increments using NumPy. This created 101 possible weight values (from 0.00 to 1.00 in steps of 0.01). To determine the number of all possible weight combinations (where the sum equals exactly 1.0) for a six-stock portfolio, we used the concept of combinations with repetition. As a result, we calculated that we had 96,560,646 possible combinations. Where n = 100 + 6 - 1 = 105 represents the total number of percentage points to be distributed and k = 6 – 1 = 5 represents the number of assets in the portfolio. And then, we created an array with the shape of (96560646, 8) that would store the data for us.

After that, we looped through all the weight combinations and computed the average monthly return of each individual portfolio by using the dot product of the weight vector with the vector of average monthly returns and computed its standard deviation (risk) using the portfolio standard deviation formula, $\sigma_p = (w^T\Omega w)^{1/2}$, where Ω is the covariance matrix of the asset returns and T transposes the weights.

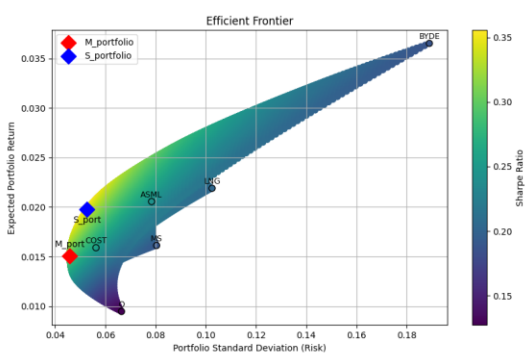
Then, we stored the weights (w) in columns 0 through 6, the expected monthly return and standard deviation of each individual portfolio to columns 7 and 8, respectively.

Finally, we calculated the Sharpe ratio for each portfolio using formula: Sharpe Ratio = $(R_p - R_f) / \sigma_p$, where R_p is the average monthly portfolio return, $R_f = 0.001$ is the assumed monthly risk-free rate, and σ_p is the portfolio standard deviation.

Question 5

```
Portfolio M (Minimum Risk):
Weights: {'BYDE': 0.02, 'COST': 0.44, 'ASML': 0.07, 'LNG': 0.05, 'MS': 0.13, 'O':
 0.29}
Expected Return: 0.015099534240170659
Volatility: 0.04580760130416218
Sharpe Ratio: 0.3077990079975994

Portfolio S (Maximum Sharpe Ratio):
Weights: {'BYDE': 0.09, 'COST': 0.48, 'ASML': 0.24, 'LNG': 0.17, 'MS': 0.0, 'O': 0.02}
Expected Return: 0.01976522735638227
Volatility: 0.05274042516602886
Sharpe Ratio: 0.3558034903455674
```



This graph illustrates the relationship between the expected return and the standard deviation (risk) for portfolios constructed from different weighting combinations of stocks. Portfolios positioned towards the left side of the graph offer lower risk levels, while portfolios on the right exhibit higher risk. Expected returns also range from the lowest at the bottom to the highest at the top.

Individual stocks are marked clearly on the plot, demonstrating their standalone risk and return characteristics. For instance, a portfolio entirely composed of BYD Company Limited (ticker: BYDE) stock yields an average monthly return of approximately 3.653%, accompanied by a relatively high standard deviation of about 18.905%, reflecting its higher risk, which gives a Sharpe ratio of 0.188. Conversely, a portfolio fully invested in stock Realty Income (ticker: O) generates a considerably lower monthly return of around 0.946%, but also carries a significantly reduced risk, indicated by a standard deviation of 6.643%. Hence, it gives a Sharpe ratio of 0.127.

However, it is evident from the graph that choosing a portfolio composed exclusively of a single stock is not optimal. Portfolios that include multiple stocks clearly provide superior Sharpe ratios, meaning that they deliver higher returns per unit of risk taken. This is likely due to the benefits of diversification, which reduces idiosyncratic risk—the unique risks associated with individual stocks. For example, portfolio M is the minimum variance portfolio which gives the lowest possible risk out of any portfolios we can feasibly create. Whereas Portfolio S maximizes the Sharpe ratio and as a result, gives us the best trade-off between risk and return, making it ideal for a risk-averse investor (who is also greedy).

According to the principle of first-order stochastic dominance, if the investors are greedy, then they will prefer portfolios that offer the highest possible return for a given level of risk. So, the investment options that are on the top of the plot would first-order stochastic dominate the portfolios on the bottom. Furthermore, according to a special case of second-order stochastic dominance, the risk averse investors favor portfolios with the lowest risk for any specific return level. As a result, the feasible set of portfolios lies on the left edge of the plot. And so, these optimal portfolios are represented on the frontier. Consequently, rational investors will always select portfolios positioned along the efficient frontier based on their mean-variance utility function, ensuring they receive the best possible return for the level of risk.

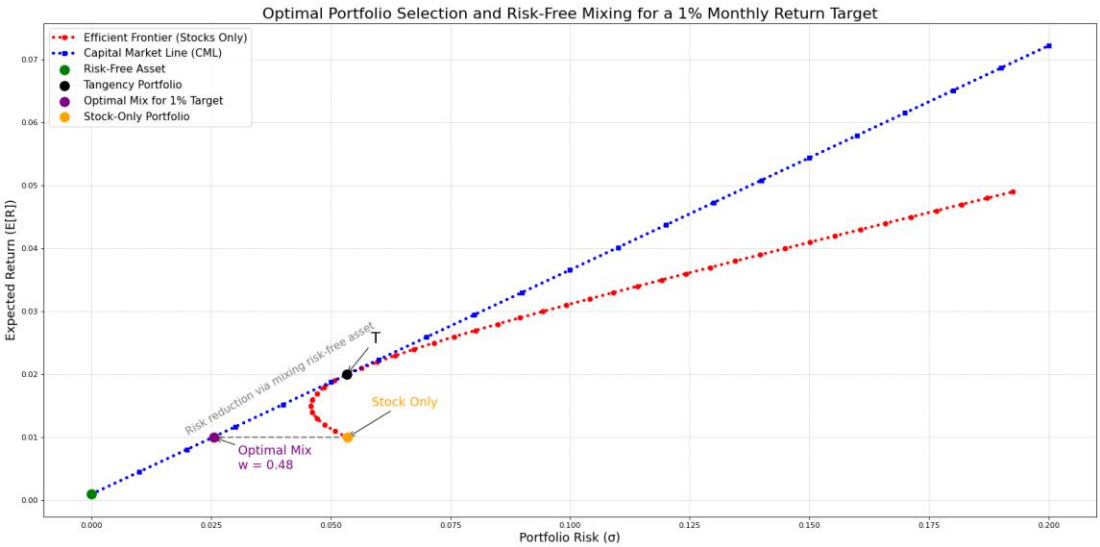
Question 6

From there, we computed both the minimum risk portfolio (M) and maximum Sharpe ratio portfolio (S).
The results can be seen from the graph for Question 5.

As shown by the graph, for the investor who simply wants to minimize their risk, with a portfolio of stocks BYDE, Costco, ASML Holding, Cheniere Energy, Morgan Stanley, and Realty Income, the weights were 0.02, 0.44, 0.07, 0.05, 0.13, 0.29, respectively. This gave us an expected monthly portfolio return of 1.510%, the monthly standard deviation of 4.581%, and a Sharpe ratio of 0.308. Intuitively, one can ascertain that in the minimum risk portfolio, most of the weighting are on stocks which are relatively stable to market fluctuations such as Realty Income and Costco, which have monthly standard deviations of around 6.643% and 5.627%, respectively (these are the least risky individual stocks in our portfolio). Whereas BYDE, on the other hand, has a relatively high standard deviation of around 18.905% (relatively high risk), which explains why it is not included in the portfolio.

On the other hand, for the investor who wants to maximize the Sharpe ratio, we recommend the portfolio of stocks BYDE, Costco, ASML Holding, Cheniere Energy, Morgan Stanley, and Realty Income with the weights of 0.09, 0.48, 0.24, 0.17, 0.0, 0.02, respectively. This gave us the portfolio with the expected monthly return of 1.977%, the monthly standard deviation of 5.274%, and a Sharpe ratio of 0.356. As is evident, most of the weighting is given to Costco followed by ASML Holding and Cheniere Energy. This is because in order to get the highest Sharpe ratio, one wants the return to be relatively high and the standard deviation to be lower. This would, as a result, provide better compensation to investors per unit of risk taken. For instance, an individual stock like Costco gives an expected monthly return of 1.590% and a relatively low monthly standard deviation of 5.627%, which results in a Sharpe ratio of 0.265 (the highest Sharpe ratio among our individual stocks). However, a stock such as Morgan Stanley, which is a financial services company, is not included at all. This can be explained by the fact that while Morgan Stanley has an expected monthly return of around 1.613%, which is relatively similar to Costco's 1.590%, it has a significantly higher standard deviation (risk), meaning that it has a smaller Sharpe ratio. Hence, Costco is better at compensating investors per unit of risk taken, relative to Morgan Stanley, which explains its higher weighting.

Question 7



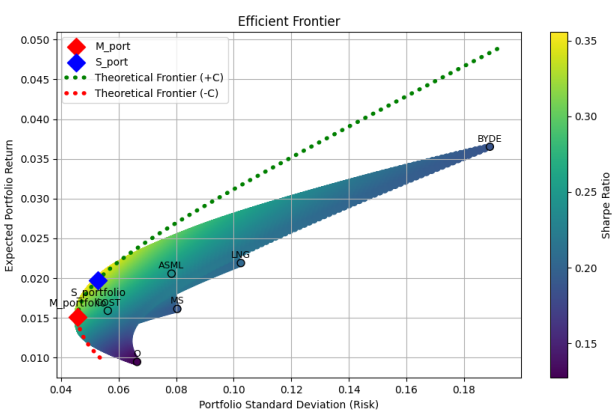
For the third investor to achieve an expected return target of 1% per month, they can create an optimal stock portfolio through a combination of the maximum Sharpe ratio portfolio (S) and a risk-free asset. So, the portfolio that maximizes the Sharpe ratio has an expected monthly return of 1.977% and monthly standard deviation of 5.274%. The weighting of portfolio S is determined such that the overall portfolio has an expected return of 1% per month.

$R_{Target} = w * R_s + (1 - w) * R_f$

$1\% = w * R_s + (1 - w) * R_f$

Thus, 1% = 0.0198w + 0.001 - 0.001w. Solving for "w," we found that the investor needs to allocate 52% of her capital in the risk-free asset (lending at the risk-free rate) and 48% into portfolio S. However, if the investor instead holds a portfolio of only stocks, they will have to choose a portfolio on the efficient frontier with an expected return of 1%, resulting in a portfolio lying below the Capital Market Line (CML). Hence, it would have a lower Sharpe ratio than the portfolio we built by combining the Portfolio S and the risk-free asset. So, for the same expected return, the investor would take on more risk if they held a portfolio of stocks only. Overall, this demonstrates the benefits of combining an equity portfolio with the risk-free asset to achieve the expected return while significantly reducing the risk taken.

<u>Question 8</u>



The portfolio frontier represents the set of portfolios which have a minimal risk for their respective levels of expected returns. To construct the efficient frontier for a set of six stocks, we compute the optimal portfolio weights that minimize risk (variance), subject to achieving a target expected return C, and ensuring that the weights sum to 1.

Mathematically, the optimization problem is formulated as:

$$\min \tfrac{1}{2} w'\Omega w$$

Subject to: $\mu' w = C$ (target expected return), $\tilde{1}' w = 1$ (weights sum should be equal to 1)

To solve this we construct Lagrangian:

$$L(w, \lambda, \gamma) = w^T\Omega w - \lambda(\mu^T w - C) - \gamma(1^T w - 1)$$

Taking the first-order conditions and solving this system of equations:

$$\partial L / \partial w = 2\Omega w - \lambda\mu - \gamma 1 = 0$$
$$\partial L / \partial \lambda = \mu^T w - C = 0$$
$$\partial L / \partial \gamma = 1^T w - 1 = 0$$

Define the constants:

$$A = 1^T\Omega^{-1}\mu$$

$$B = \mu^T\Omega^{-1}\mu$$

$$F = 1^T\Omega^{-1}1$$

$$D = BF - A^2$$

Then the optimal portfolio weights for a given target return C are:

$$w(C) = \alpha_0 + \alpha_1 \cdot C$$

Where:

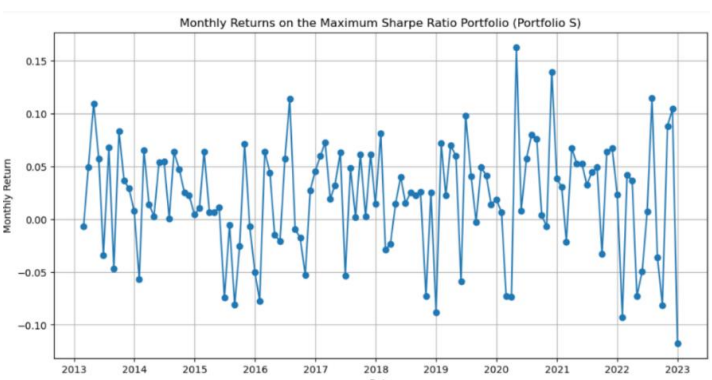
$$\alpha_0 = (1 / D) * [B\Omega^{-1}1 - A\Omega^{-1}\mu]$$
$$\alpha_1 = (1 / D) * [F\Omega^{-1}\mu - A\Omega^{-1}1]$$

We then varied the target return $C \in [0.01, 0.05]$ in increments of 0.001 and computed the portfolio standard deviation for each return:

$$\sigma_p = \sqrt{(w(C)^T\Omega w(C))}$$

The resulting green curve on the efficient frontier represents the set of portfolios that dominate others by offering higher expected returns for the same level of risk, or equivalently, lower risk for the same level of return.

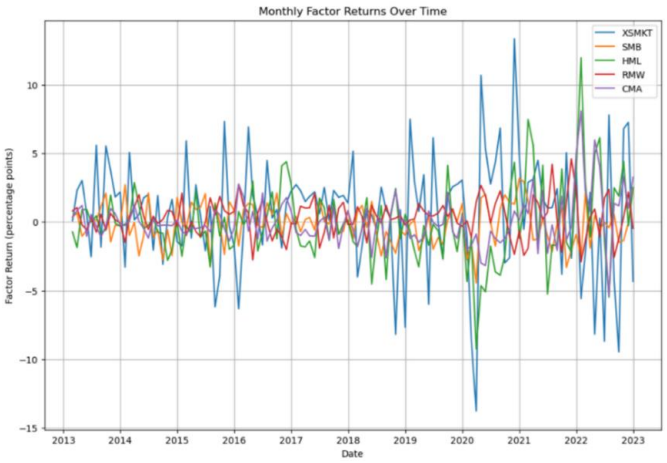
<u>Question 9</u>




```
Portfolio S Monthly Returns - Summary Statistics:
count    119.000000
mean       0.019765
std        0.052740
min       -0.117503
25%       -0.007244
50%        0.024924
75%        0.057270
max        0.162601
dtype: float64
Skewness: -0.2748899870361411
Kurtosis: 0.0023367745345539426
```

The maximum Sharpe ratio portfolio (Portfolio S) has a monthly expected return of around 2%, indicating moderate positive performance, and a monthly standard deviation of around 5.274% which shows a moderate level of variability from the mean, which was computed based on 10 years of data. The worst-performing month had a return of -11.75%, indicating exposure to downside risk, while the best month yielded a 16.26% gain, showing that the portfolio has also experienced strong positive surges. The skewness of -

0.275 suggests that the returns are slightly left-skewed, indicating a small tendency toward more extreme negative returns. The near-zero kurtosis of 0.002 indicates that there are not any significant fat tails or excess peaks.



Factor Returns - Summary Statistics:

	XSMKT	SMB	HML	RMW	CMA
count	119.000000	119.000000	119.000000	119.000000	119.000000
mean	0.706975	-0.123950	-0.110588	0.315546	0.051345
std	4.204784	1.480848	2.820208	1.317181	1.753325
min	-13.770000	-4.440000	-9.240000	-2.910000	-5.360000
25%	-1.735000	-1.105000	-1.705000	-0.395000	-0.960000
50%	1.040000	-0.100000	-0.490000	0.410000	-0.230000
75%	2.750000	1.015000	1.185000	1.080000	0.795000
max	13.340000	3.160000	11.960000	4.590000	8.090000

By looking at the summary statistics for the factor returns based on 10 years of sample data, the excess market return (XSMKT) has a positive mean excess return of 0.707 and a standard deviation of 4.2, the highest among the factors, suggesting that the market factor is the most volatile. Meanwhile, on average, large-cap stocks slightly outperformed small-caps during this period and the SMB factor has a standard deviation of 1.48 making it less volatile than the market factor. Also, the value factor (HML)'s summary statistics indicate that growth stocks marginally outperformed value stocks (mean: -0.111). Moreover, the value factor (HML) is right-skewed, with extreme positive outliers. Whereas, the profitability factor (RMW) has a **positive average return of 0.316**, suggesting that **profitable firms consistently outperformed less profitable ones,** and it has a relative low volatility. Lastly, the investment factor (CMA) has a mean of 0.051, which indicates a small but positive return premium for conservative investment firms and it has a moderate volatility.

```
Breusch-Godfrey Test Results:
LM Statistic: 11.806455637778614
LM p-value: 0.4613460830156998
F Statistic: 0.9270241245329315
F p-value: 0.5233249999169636

ARCH test Results:
LM Statistic: 12.058180497385454
LM p-value: 0.441018474116547
F Statistic: 0.9948803143267988
F p-value: 0.45992540170497587
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:       Portfolio_Return   R-squared:                       0.704
Model:                            OLS   Adj. R-squared:                  0.691
Method:                 Least Squares   F-statistic:                     53.68
Date:                Mon, 07 Apr 2025   Prob (F-statistic):           2.79e-28
Time:                        16:04:08   Log-Likelihood:                 254.17
No. Observations:                 119   AIC:                            -496.3
Df Residuals:                     113   BIC:                            -479.7
Df Model:                           5
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          0.0104      0.003      3.696      0.000       0.005       0.016
XSMKT          0.0103      0.001     14.290      0.000       0.009       0.012
SMB           -0.0035      0.002     -1.748      0.083      -0.007       0.000
HML           -0.0029      0.002     -1.464      0.146      -0.007       0.001
RMW            0.0039      0.003      1.429      0.156      -0.001       0.009
CMA            0.0016      0.003      0.562      0.575      -0.004       0.007
==============================================================================
Omnibus:                        1.376   Durbin-Watson:                   2.096
Prob(Omnibus):                  0.503   Jarque-Bera (JB):                0.969
Skew:                           0.201   Prob(JB):                        0.616
Kurtosis:                       3.183   Cond. No.                         5.89
==============================================================================
```

Our analysis of the regression started from considering the p-value of the f-statistic, which was near zero (2.79e-28), meaning that we could reject the null hypothesis. This implies that the model that we have constructed is statistically significant in explaining the monthly return of the maximum Sharpe ratio portfolio on the given factors.

Before we could analyze the model further, we had to first determine if the Gauss-Markov assumptions have been violated. Specifically, whether there was heteroscedasticity and serial correlation present in the model.[1] Given that we were working with time-series data, we conducted the Breusch-Godfrey test for serial correlation and an ARCH test for heteroscedasticity.[2] Assuming a significance level of 5%, we could not confidently reject the null hypotheses given that the p-values provided by the tests for heteroscedasticity and serial correlation were around 0.523 and 0.460, respectively (all > 0.05). This meant that there was neither serial correlation nor heteroscedasticity in the model, and therefore, no corrections for the standard errors were required before interpreting the regression results.

From there, we analyzed the statistical results we got for our explanatory variables. We found that we could only reject the null hypothesis for the constant (the alpha, or abnormal return) and the excess return on the market factor (XSMKT), which suggests that the alpha and market factor are significant in explaining the expected value of the portfolio's return. However, in the cases of the size (SMB), value (HML), profitability (RNW), and investment (CMA) factors, they are not statistically significant at the 5% level, which indicates that the portfolio (S)'s expected monthly returns cannot consistently be explained by these factors.

Therefore, we had to drop the least non-significant predictor (CMA) and re-estimate the regression model. The results we got showed us that the p-value of the (RMW) was still non-significant, so we repeated this process again. The results are presented in the table below:

```
                            OLS Regression Results
==============================================================================
Dep. Variable:      Portfolio Returns   R-squared:                       0.696
Model:                            OLS   Adj. R-squared:                  0.688
Method:                 Least Squares   F-statistic:                     87.69
Date:                Tue, 08 Apr 2025   Prob (F-statistic):           1.38e-29
Time:                        16:43:34   Log-Likelihood:                 252.60
No. Observations:                 119   AIC:                            -497.2
Df Residuals:                     115   BIC:                            -486.1
Df Model:                           3
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          0.0116      0.003      4.225      0.000       0.006       0.017
XSMKT          0.0102      0.001     15.674      0.000       0.009       0.011
SMB           -0.0047      0.002     -2.539      0.012      -0.008      -0.001
HML           -0.0030      0.001     -3.094      0.002      -0.005      -0.001
==============================================================================
Omnibus:                        2.586   Durbin-Watson:                   2.155
Prob(Omnibus):                  0.274   Jarque-Bera (JB):                2.113
Skew:                           0.313   Prob(JB):                        0.348
Kurtosis:                       3.188   Cond. No.                         4.36
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

All three factors (XSMKT, SMB, HML) are statistically significant, reinforcing that they each provide meaningful insights into the portfolio's return structure.

The intercept (or alpha) in the regression model indicates that the portfolio earned an average monthly excess return of 1.16%, beyond what would be expected based on its exposure to the market, size, and value factors. This positive and statistically significant alpha may reflect managerial skill, market inefficiencies, or the impact of omitted risk factors.

In addition, the coefficient on XSMKT, representing the excess return on the market, is 0.0102, suggesting that for every 1% increase in the market's excess return, the portfolio's return increases by approximately 1.02%, *ceteris paribus*. This implies that the portfolio is highly sensitive to overall market performance, and is therefore likely to perform well in bull markets, but it may also lead to greater downside risk during market downturns.

In contrast, the SMB (Small Minus Big) factor has a negative coefficient of -0.0047. This means that a 1% increase in SMB results in a 0.47% decrease in portfolio returns, holding other variables constant. Therefore, when small caps outperform large caps, portfolio S underperforms.

Similarly, the coefficient on the HML (High Minus Low) factor is -0.0030, suggesting a tilt toward growth stocks rather than value. A 1% increase in HML is associated with a 0.30% decrease in the portfolio's return, ceteris paribus. Thus, the portfolio is likely to underperform in markets favoring value-oriented investing, whereas it may benefit more when growth stocks dominate.

---

[1] We do not need to worry about the assumption that the errors have zero mean being violated because we have a constant in the regression.

[2] Given that we are working with monthly data as well, for each test, we will use 12 lags to ensure their accuracy.

In summary, a significant portion of the portfolio's risk comes from market risk. So a decline in the overall market would result in a relatively large decrease in the portfolio's value. Hence, Investors might consider hedging against market downturns.

Appendix A: Python Script:

```
from google.colab import drive

drive.mount('/content/drive')

import pandas as pd

import matplotlib.pyplot as plt

import numpy as np

df_stocksall = pd.read_excel("/content/drive/MyDrive/FR2209_assessment_data.xlsx")

df_stocksall.head()

selection = np.array(["O", "LNG", "MS", "COST", "BYDE", "ASML"])


# Filter columns

df_stocksprice = df_stocksall.loc[:, df_stocksall.columns.isin(selection)]

print(df_stocksprice)

df_stocksret = (df_stocksprice / df_stocksprice.shift(1) - 1).dropna()

df_stocksret

df_date = df_stocksall.loc[:, df_stocksall.columns=="date"]

print(df_date)
```

```
df_stocks = pd.concat([df_date, df_stocksret], axis=1).dropna()

#df_stocks = pd.merge(df_date, df_stocksret , on="Date", how="inner") # 'inner', 'left', 'right', or 'outer'

print(df_stocks)
type(df_stocks.loc[1,'date'])
mu = df_stocks[["O", "LNG", "MS", "COST", "BYDE", "ASML"]].mean(axis=0)
plt.figure(figsize=(20, 10), dpi=80)
plt.plot(df_stocks['date'], df_stocks['O'], label="Realty Income Return")

# Draw horizontal lines
plt.axhline(y=mu["O"], color='grey', linestyle='--', linewidth=1, label='Mean')
plt.axhline(y=0, color='black', linestyle='--', linewidth=1, label='Zero')

# Annotate the mean line
plt.text(df_stocks['date'].iloc[-1], mu["O"], f'Mean: {mu["O"]:.4f}',
va='bottom', ha='right', fontsize=12, color='black')

plt.legend()
plt.xlabel('Date')
plt.ylabel('Realty Income (ticker: O)')
plt.title('Realty Income (ticker: O) Return')
plt.savefig("O._returns.pdf")

plt.figure(figsize=(20, 10), dpi=80)
plt.plot(df_stocks['date'], df_stocks['MS'], label="Morgan Stanley Return")
```

```
# Draw horizontal lines
plt.axhline(y=mu["MS"], color='grey', linestyle='--', linewidth=1, label='Mean')
plt.axhline(y=0, color='black', linestyle='--', linewidth=1, label='Zero')

# Annotate the mean line
plt.text(df_stocks['date'].iloc[-1], mu["MS"], f'Mean: {mu["MS"]:.4f}',
va='bottom', ha='right', fontsize=12, color='black')

plt.legend()
plt.xlabel('Date')
plt.ylabel('Morgan Stanley (ticker: MS)')
plt.title('Morgan Stanley (ticker: MS) Return')
plt.savefig("ms.pdf")

plt.figure(figsize=(20, 10), dpi=80)
plt.plot(df_stocks['date'], df_stocks['ASML'], label="ASML Holding Return")

# Draw horizontal lines
plt.axhline(y=mu["ASML"], color='grey', linestyle='--', linewidth=1, label='Mean')
plt.axhline(y=0, color='black', linestyle='--', linewidth=1, label='Zero')

# Annotate the mean line
plt.text(df_stocks['date'].iloc[-1], mu["ASML"], f'Mean: {mu["ASML"]:.4f}',
va='bottom', ha='right', fontsize=12, color='black')
```

```
plt.legend()
plt.xlabel('Date')
plt.ylabel('ASML Holding (ticker: ASML)')
plt.title('ASML Holding (ticker: ASML)')
plt.savefig("ASML.pdf")
plt.figure(figsize=(20, 10), dpi=80)
plt.plot(df_stocks['date'], df_stocks['BYDE'], label="BYD Company Limited Return")

# Draw horizontal lines
plt.axhline(y=mu["BYDE"], color='grey', linestyle='--', linewidth=1, label='Mean')
plt.axhline(y=0, color='black', linestyle='--', linewidth=1, label='Zero')

# Annotate the mean line
plt.text(df_stocks['date'].iloc[-1], mu["BYDE"], f'Mean: {mu["BYDE"]:.4f}',
va='bottom', ha='right', fontsize=12, color='black')

plt.legend()
plt.xlabel('Date')
plt.ylabel('BYD Company Limited (ticker: BYDE)')
plt.title('BYD Company Limited (ticker: BYDE)')
plt.savefig("BYDE.pdf")
plt.figure(figsize=(20, 10), dpi=80)
plt.plot(df_stocks['date'], df_stocks['COST'], label="CostCo Return")

# Draw horizontal lines
plt.axhline(y=mu["COST"], color='grey', linestyle='--', linewidth=1, label='Mean')
```

```
plt.axhline(y=0, color='black', linestyle='--', linewidth=1, label='Zero')

# Annotate the mean line
plt.text(df_stocks['date'].iloc[-1], mu["COST"], f'Mean: {mu["COST"]:.4f}',
va='bottom', ha='right', fontsize=12, color='black')

plt.legend()
plt.xlabel('Date')
plt.ylabel('CostCo (ticker: COST)')
plt.title('CostCo (ticker: COST)')
plt.savefig("Cost.pdf")
plt.figure(figsize=(20, 10), dpi=80)
plt.plot(df_stocks['date'], df_stocks['LNG'], label="Cheniere Energy Inc Return")

# Draw horizontal lines
plt.axhline(y=mu["LNG"], color='grey', linestyle='--', linewidth=1, label='Mean')
plt.axhline(y=0, color='black', linestyle='--', linewidth=1, label='Zero')

# Annotate the mean line
plt.text(df_stocks['date'].iloc[-1], mu["LNG"], f'Mean: {mu["LNG"]:.4f}',
va='bottom', ha='right', fontsize=12, color='black')

plt.legend()
plt.xlabel('Date')
plt.ylabel('Cheniere Energy Inc (ticker: LNG)')
plt.title('Cheniere Energy Inc (ticker: LNG)')
```

```
plt.savefig("LNG.pdf")

stats_df = df_stocks.describe().transpose()

stats_df.rename(columns={"count": "N", "mean": "Mean", "std": "St. Dev.", "25%": "Pctl(25)","75%": "Pctl(75)", "50%": "Median"}, inplace=True)

stats_df = stats_df.iloc[:,[0,1,2,4,5,6]]

print("Return descriptive stats\n")

print(round(stats_df,1))


# Calculate the mean (average) return for each selected stock

mu = df_stocks[["O", "LNG", "MS", "COST", "BYDE", "ASML"]].mean(axis=0)


# Calculate the standard deviation (volatility) of returns for each selected stock

std_dev = df_stocks[["O", "LNG", "MS", "COST", "BYDE", "ASML"]].std(axis=0)

print(std_dev)

R_f = 0.001

#Calculate the Sharpe ratio for each stock

sharpe_ratios = (mu - R_f) / std_dev

# Print the Sharpe ratios

print(sharpe_ratios)

# Create a DataFrame with mu, std_dev, and Sharpe ratios

portfolio_stats = pd.DataFrame({

'mu': mu,

'std_dev': std_dev,

'sharpe_ratios': sharpe_ratios

})

portfolio_stats

# Calculate the covariance matrix for the selected stocks
```

```
Omega = df_stocks[["O", "LNG", "MS", "COST", "BYDE", "ASML"]].cov()

# Calculate the correlation matrix for the selected stocks
rho = df_stocks[["O", "LNG", "MS", "COST", "BYDE", "ASML"]].corr()
import seaborn as sns

# Creating the correlation matrix
plt.figure(figsize=(10, 8))
sns.heatmap(rho, annot=True, cmap='coolwarm', fmt=".2f")
plt.title('Correlation Matrix of Stock Returns')
plt.show()

# Mean of equally weighted portfolio

w = np.array([1/6,1/6,1/6,1/6,1/6,1/6])
mu_p = np.dot(w.transpose(),mu)
var_p = np.dot(np.dot(w.transpose(),Omega),w) #Finding the variance of the portfolio
# Our portfolio
#["O", "LNG", "MS", "COST", "BYD", "ASML"]
w = np.array([0.23,0.17,0.15,0.10,0.2,0.15])
mu_p = np.dot(w.transpose(),mu)
var_p = np.dot(np.dot(w.transpose(),Omega),w)
# Calculate the Sharpe ratio for our portfolio
sharpe_ratio_portfolio = (mu_p - R_f) / np.sqrt(var_p)

print(f"Sharpe Ratio of the portfolio: {sharpe_ratio_portfolio}")
```

```
# Create a DataFrame for the portfolio statistics

portfolio_summary = pd.DataFrame({

'Metric': ['Portfolio Return (mu)', 'Portfolio Variance (var_p)', 'Portfolio Standard Deviation (sqrt(var_p))',
'Sharpe Ratio'],

'Value': [mu_p, var_p, np.sqrt(var_p), sharpe_ratio_portfolio]

})


# Display the DataFrame

print(portfolio_summary)

# Rough Mean Variance Frontier

# n = 100+6-1 k=6-1=5

import math

x = math.comb(105,5)

print(x)

resMat = np.empty((96560646,8))

resMat[:] = np.nan

resMat


resMat.shape


mySeq = np.arange(0,101)


# Generate all possible portfolio weight combinations (that sum to 1) based on values in mySeq,

# and calculate the expected return and risk (standard deviation) for each portfolio

myCount = 0

for i in mySeq:

tmp = 100 - i
```

```
print(i)

for j in range(tmp + 1):

k = 100 - i - j

for l in range(k + 1):

m = 100 - i - j - l

for n in range(m + 1):

o = 100 - i - j - l - n

for p in range(o + 1):

q = 100 - i - j - l - n - p


# Normalize weights to sum to 1

w = np.array([i, j, l, n, p, q]) / 100

# Store portfolio weights

resMat[myCount, 0:6] = w.transpose()

# Calculate and store expected portfolio return

resMat[myCount, 6] = np.dot(w.transpose(), mu)

# Calculate and store portfolio standard deviation (risk)

resMat[myCount, 7] = (np.dot(np.dot(w.transpose(), Omega), w)) ** 0.5

# Move to next portfolio combination

myCount = myCount+1



resData = pd.DataFrame(resMat,
columns=["w1_O","w2_LNG","w3_MS","w4_Cost","w5_BYDE","w6_ASML","mu_p", "sigma_p"])


#the scatter graph of resData mu_p on sigma_p
```

```
plt.figure(figsize=(10, 6))
plt.scatter(resData['sigma_p'], resData['mu_p'])
plt.xlabel('Portfolio Volatility (sigma_p)')
plt.ylabel('Portfolio Return (mu_p)')
plt.title('Portfolio Volatility vs. Return')
plt.grid(True)
plt.show()


print(f"Log Risk-Free Rate: {R_f:.6f}")

sharpe_ratios = []

for i in range(len(resData)): # Iterate over all rows
muOfP = resData.loc[i, 'mu_p']
sigmaOfP = resData.loc[i, 'sigma_p']

if sigmaOfP != 0: # Avoid division by zero
sharpe_ratio = (muOfP - R_f) / sigmaOfP
else:
sharpe_ratio = None # Assign None (or float('nan')) if sigma is zero

sharpe_ratios.append(sharpe_ratio)

# Create DataFrame after the loop
df_sharpe = pd.DataFrame({'Sharpe Ratio': sharpe_ratios})
```

```
# Print results
print(df_sharpe)
```

```
# # Combine the DataFrames
df_portfolios = pd.concat([resData, df_sharpe], axis=1)

# Print the combined DataFrame
df_portfolios

# Sort the data in the df_portfolios from the highest to lowest of the sharp ratio
# Portfolio S
# Sort the DataFrame by the 'Sharpe Ratio' column in descending order
df_portfolios_sorted_S = df_portfolios.sort_values(by='Sharpe Ratio', ascending=False)

# Display the sorted DataFrame
df_portfolios_sorted_S

# Sort the data in the df_portfolios from the lowest to highest the Risk (Standard deviation)
# Portfolio M
# Sort the DataFrame by the 'Sharpe Ratio' column in descending order
df_portfolios_sorted_M = df_portfolios.sort_values(by='sigma_p', ascending=True)
```

```
# Display the sorted DataFrame
df_portfolios_sorted_M

S_port = df_portfolios_sorted_S.iloc[0]

print("An investor who wants to maximise Sharpe ratio. Portfolio S:")
print(S_port)

M_port = df_portfolios_sorted_M.iloc[0]

print("An investor who simply wants to minimise risk. Portfolio M:")
print(M_port)

#Visualisation
import numpy as np
import pandas as pd
import itertools
import matplotlib.pyplot as plt
selected_columns = ["w1_O","w2_LNG","w3_MS","w4_Cost","w5_BYDE","w6_ASML"]
portfolios = np.array(df_portfolios[selected_columns])
print(portfolios)

# Convert lists to numpy arrays for easy plotting
expected_returns = np.array(df_portfolios['mu_p'])
print(expected_returns)
portfolio_std_devs = np.array(df_portfolios['sigma_p'])
```

```
print(portfolio_std_devs)
sharpe_ratios = np.array(df_portfolios['Sharpe Ratio'])
print(sharpe_ratios)

portfolio_df = df_portfolios
print(portfolio_df.head())

# Plotting the efficient frontier (Expected return vs Portfolio Standard Deviation)
plt.figure(figsize=(10, 6))
plt.scatter(portfolio_std_devs, expected_returns, c=sharpe_ratios, cmap='viridis', marker='o')
plt.colorbar(label='Sharpe Ratio') # Color bar to represent Sharpe ratio
plt.xlabel('Portfolio Standard Deviation (Risk)')
plt.ylabel('Expected Portfolio Return')
plt.title('Efficient Frontier')

# Add stock names to the points where the weight is 1 for each stock
for i, stock in enumerate(mu.index): # mu.index contains the stock names
# Find the portfolios where stock i has weight 1
stock_weights = portfolios[:, i]
for j, weight in enumerate(stock_weights):
if weight == 1:
# Annotate the plot with only the stock name (no mean return)
plt.annotate(f'{stock}', (portfolio_std_devs[j], expected_returns[j]),
textcoords="offset points", xytext=(0, 5), ha='center', fontsize=9)
# Plot the point with a black border, same size as other points, no red filling
plt.scatter(portfolio_std_devs[j], expected_returns[j], c='none', edgecolors='black', s=50, zorder=5)
```

```
plt.grid(True)
plt.show()

# Optionally, print the best portfolio (highest Sharpe ratio)
max_sharpe_idx = np.argmax(sharpe_ratios)
print(f"Best portfolio (highest Sharpe ratio):\nExpected Return: {expected_returns[max_sharpe_idx]}")
print(f"Standard Deviation: {portfolio_std_devs[max_sharpe_idx]}")
print(f"Sharpe Ratio: {sharpe_ratios[max_sharpe_idx]}")


# Plotting the efficient frontier (Expected return vs Portfolio Standard Deviation)
plt.figure(figsize=(10, 6))
plt.scatter(portfolio_std_devs, expected_returns, c=sharpe_ratios, cmap='viridis', marker='o')
plt.colorbar(label='Sharpe Ratio') # Color bar to represent Sharpe ratio
plt.xlabel('Portfolio Standard Deviation (Risk)')
plt.ylabel('Expected Portfolio Return')
plt.title('Efficient Frontier')

# Add stock names to the points where the weight is 1 for each stock
for i, stock in enumerate(mu.index): # mu.index contains the stock names
# Find the portfolios where stock i has weight 1
stock_weights = portfolios[:, i]
for j, weight in enumerate(stock_weights):
if weight == 1:
# Annotate the plot with only the stock name (no mean return)
```

```
plt.annotate(f'{stock}', (portfolio_std_devs[j], expected_returns[j]),
textcoords="offset points", xytext=(0, 5), ha='center', fontsize=9)
# Plot the point with a black border
plt.scatter(portfolio_std_devs[j], expected_returns[j], c='none', edgecolors='black', s=50, zorder=5)

# -------------------------------
# Annotate the special portfolios

# Example: Annotate the Market Portfolio (M_port)
# Assuming M_port has an expected return 'mu_M' and standard deviation 'sigma_M'
plt.scatter(M_port['sigma_p'], M_port['mu_p'], marker='D', s=150, color='red', zorder=10, label='M_port')
plt.annotate('M_portfolio', (M_port['sigma_p'], M_port['mu_p']),
textcoords="offset points", xytext=(0, 10), ha='center', fontsize=10)

# Example: Annotate the Second Portfolio (S_port)
# Assuming S_port has an expected return 'mu_S' and standard deviation 'sigma_S'
plt.scatter(S_port['sigma_p'], S_port['mu_p'], marker='D', s=150, color='blue', zorder=10, label='S_port')
plt.annotate('S_portfolio', (S_port['sigma_p'], S_port['mu_p']),
textcoords="offset points", xytext=(0, -15), ha='center', fontsize=10)

# -------------------------------

plt.grid(True)
plt.legend()
plt.show()
```

```
# Optionally, print the best portfolio (highest Sharpe ratio)
max_sharpe_idx = np.argmax(sharpe_ratios)
print(f"Best portfolio (highest Sharpe ratio):\nExpected Return: {expected_returns[max_sharpe_idx]}")
print(f"Standard Deviation: {portfolio_std_devs[max_sharpe_idx]}")
print(f"Sharpe Ratio: {sharpe_ratios[max_sharpe_idx]}")

#Mathematical Frontier
mu = df_stocks[["O", "LNG", "MS", "COST", "BYDE", "ASML"]].mean(axis=0)
std_dev = df_stocks[["O", "LNG", "MS", "COST", "BYDE", "ASML"]].std(axis=0)
Omega = df_stocks[["O", "LNG", "MS", "COST", "BYDE", "ASML"]].cov()

onesVec = np.empty((1,6))
onesVec[:] = 1
onesVec

print(Omega)
print(mu)
print(onesVec)

A = np.dot(np.dot(onesVec,np.linalg.inv(Omega)),mu)[0]
B = np.dot(np.dot(mu.transpose(),np.linalg.inv(Omega)),mu)
F = float(np.dot(np.dot(onesVec,np.linalg.inv(Omega)),onesVec.transpose()))
D = B*F-A**2
alpha_0 = (np.dot((B*np.linalg.inv(Omega)),onesVec.transpose()).reshape(6) -
np.dot((A*np.linalg.inv(Omega)),mu.transpose()))/D
alpha_1 = (np.dot((F*np.linalg.inv(Omega)),mu) -
np.dot((A*np.linalg.inv(Omega)),onesVec.transpose().reshape(6)))/D
```

```
rbarVec = np.arange(0.01,0.05,0.001) #rbar is the expected return target

print(rbarVec) #These are all different C

#So for each of these target returns i can create a coreesponding portfolio with the weight and find the standard deviation


stdVec = np.empty((len(rbarVec),1))

stdVec[:] = np.nan

# print(stdVec)


for i in range(len(rbarVec)):

rbar = rbarVec[i] # This is the expected return

w = alpha_0 + alpha_1*rbar # The optimal portfolio weights for an investor

#trying to hit an expected return target of C with minimal risk

print(w)

stdVec[i] = (np.dot(np.dot(w.transpose(),Omega),w))**0.5


# Duplicated code

weights_list = []

std_list = []


for i in range(len(rbarVec)):

rbar = rbarVec[i]

w = alpha_0 + alpha_1 * rbar # Compute optimal weights

std = (np.dot(np.dot(w.T, Omega), w))**0.5 # Portfolio standard deviation

weights_list.append(w.flatten()) # Flatten in case w is a 2D array

std_list.append(std)
```

```
print(f"r̄ = {rbar:.4f}, Weights = {w.flatten()}, Std Dev = {std:.4f}")

# Convert to DataFrame
df_results_MF = pd.DataFrame(weights_list, columns=[f'Weight_{i+1}' for i in range(len(weights_list[0]))])
df_results_MF['Expected Return'] = rbarVec
df_results_MF['Standard Deviation'] = std_list

# Print final DataFrame
print("\nFinal Results:")
print(df_results_MF)

stdVec.shape

theoryData_dict = {"C":list(rbarVec), "sigma_P":list(stdVec.reshape(40,))}
theoryData = pd.DataFrame(theoryData_dict)

print(M_port["mu_p"])
theoryData_negC = theoryData[theoryData['C']< M_port["mu_p"]]
theoryData_posC = theoryData[theoryData['C'] >= M_port["mu_p"]]

plt.figure(figsize=(20, 10), dpi=80)
plt.plot(theoryData_posC['sigma_P'], theoryData_posC['C'], color='g', dash_capstyle = 'round',linestyle = (0,(0.1,2)),linewidth = 4)
plt.plot(theoryData_negC['sigma_P'], theoryData_negC['C'], color='r', dash_capstyle = 'round',linestyle = (0,(0.1,2)),linewidth = 4)
plt.xlabel('Portfolio sigma')
plt.ylabel('Portfolio mu')
```

```
plt.savefig("6stocks_theoretical_frontier.pdf")


plt.figure(figsize=(12, 8), dpi=100)


# Scatter plot of the portfolios
plt.scatter(df_portfolios['sigma_p'], df_portfolios['mu_p'], label='Portfolios', alpha=1)


# Plot the theoretical frontiers
plt.plot(theoryData_posC['sigma_P'], theoryData_posC['C'], color='g',
dash_capstyle='round', linestyle=(0, (0.1, 2)), linewidth=4, label='Theoretical Frontier (+C)')
plt.plot(theoryData_negC['sigma_P'], theoryData_negC['C'], color='r',
dash_capstyle='round', linestyle=(0, (0.1, 2)), linewidth=4, label='Theoretical Frontier (-C)')


# Labels and title
plt.xlabel('Portfolio Volatility (sigma_p)')
plt.ylabel('Portfolio Return (mu_p)')
plt.title('Portfolio Volatility vs. Return with Theoretical Frontier')
plt.grid(True)
plt.legend()


# Save and show
plt.show()



import numpy as np
import pandas as pd
```

```
import itertools
import matplotlib.pyplot as plt
selected_columns = ["w1_O","w2_LNG","w3_MS","w4_Cost","w5_BYDE","w6_ASML"]
portfolios = np.array(df_portfolios[selected_columns])
print(portfolios)

# Convert lists to numpy arrays for easy plotting
expected_returns = np.array(df_portfolios['mu_p'])
print(expected_returns)
portfolio_std_devs = np.array(df_portfolios['sigma_p'])
print(portfolio_std_devs)
sharpe_ratios = np.array(df_portfolios['Sharpe Ratio'])
print(sharpe_ratios)

portfolio_df = df_portfolios
portfolio_df.head()



# Plotting the efficient frontier (Expected return vs Portfolio Standard Deviation)
plt.figure(figsize=(10, 6))
plt.scatter(portfolio_std_devs, expected_returns, c=sharpe_ratios, cmap='viridis', marker='o')
plt.colorbar(label='Sharpe Ratio') # Color bar to represent Sharpe ratio
plt.xlabel('Portfolio Standard Deviation (Risk)')
```

```
plt.ylabel('Expected Portfolio Return')
plt.title('Efficient Frontier')

# Add stock names to the points where the weight is 1 for each stock
for i, stock in enumerate(mu.index): # mu.index contains the stock names
# Find the portfolios where stock i has weight 1
stock_weights = portfolios[:, i]
for j, weight in enumerate(stock_weights):
if weight == 1:
# Annotate the plot with only the stock name (no mean return)
plt.annotate(f'{stock}', (portfolio_std_devs[j], expected_returns[j]),
textcoords="offset points", xytext=(0, 5), ha='center', fontsize=9)
# Plot the point with a black border
plt.scatter(portfolio_std_devs[j], expected_returns[j], c='none', edgecolors='black', s=50, zorder=5)

# -------------------------------
# Annotate the special portfolios

# Example: Annotate the Market Portfolio (M_port)
# Assuming M_port has an expected return 'mu_M' and standard deviation 'sigma_M'
plt.scatter(M_port['sigma_p'], M_port['mu_p'], marker='D', s=150, color='red', zorder=10, label='M_port')
plt.annotate('M_portfolio', (M_port['sigma_p'], M_port['mu_p']),
textcoords="offset points", xytext=(0, 10), ha='center', fontsize=10)

# Example: Annotate the Second Portfolio (S_port)
# Assuming S_port has an expected return 'mu_S' and standard deviation 'sigma_S'
```

```
plt.scatter(S_port['sigma_p'], S_port['mu_p'], marker='D', s=150, color='blue', zorder=10, label='S_port')
plt.annotate('S_portfolio', (S_port['sigma_p'], S_port['mu_p']),
textcoords="offset points", xytext=(0, -15), ha='center', fontsize=10)

# ----------------------------------------------
# Plot the theoretical frontiers on the same graph
# ----------------------------------------------
plt.plot(theoryData_posC['sigma_P'], theoryData_posC['C'],
color='g', dash_capstyle='round',
linestyle=(0, (0.1, 2)), linewidth=4, label='Theoretical Frontier (+C)')
plt.plot(theoryData_negC['sigma_P'], theoryData_negC['C'],
color='r', dash_capstyle='round',
linestyle=(0, (0.1, 2)), linewidth=4, label='Theoretical Frontier (-C)')

# ------------------------------

plt.grid(True)
plt.legend()
plt.savefig("6stocks_efficient_frontier.pdf")
plt.show()


#Include a risk free asset and find the CML
rf = 0.001
theoryData
maxSharpe = max((theoryData["C"]-rf)/theoryData['sigma_P'])
```

```
print(maxSharpe)

tangencyIdx = ((theoryData['C']-rf)/theoryData['sigma_P']).idxmax()

theoryData.loc[tangencyIdx,:]

theoryData.loc[tangencyIdx,"C"]

theoryData.loc[tangencyIdx,"sigma_P"]

graphData = theoryData

graphData.rename(columns={"C": "Frontier"},inplace=True)

graphData.head()

graphData['variable'] = "Frontier"

graphData.rename(columns={"Frontier": "value"},inplace=True)

graphData.head()

tmp = np.arange(0,0.21,0.01)

tmp2 = rf+tmp*maxSharpe


tmpframe_dict = {"sigma_P":tmp,"value":tmp2}

tmpframe = pd.DataFrame(tmpframe_dict)

tmpframe['variable'] = 'CML'

tmpframe.head()


print(graphData)


graphData = pd.concat([tmpframe, graphData])

graphData.shape

graphData_f = graphData[graphData['variable']=='Frontier']

graphData_c = graphData[graphData['variable']=='CML']
```

```
plt.figure(figsize=(20, 10), dpi=80)

plt.plot(graphData_f['sigma_P'], graphData_f['value'], color='red', dash_capstyle = 'round',linestyle = (0,(0.1,2)),linewidth = 4,label = "Frontier")

plt.plot(graphData_c['sigma_P'], graphData_c['value'], color='blue', dash_capstyle = 'round',linestyle = (0,(0.1,2)),linewidth = 4, label = "CML")

plt.legend()


theoryData.loc[tangencyIdx,"value"]



# Annotation UU.

plt.annotate("T",

xy=(theoryData.loc[tangencyIdx,"sigma_P"],theoryData.loc[tangencyIdx,"value"]), # arrow position (x, y)

xytext=(theoryData.loc[tangencyIdx,"sigma_P"],theoryData.loc[tangencyIdx,"value"]), # text position (x, y)

color='black',

fontsize=20)


plt.annotate("●",

xy=(theoryData.loc[tangencyIdx,"sigma_P"],theoryData.loc[tangencyIdx,"value"]), # arrow position (x, y)

xytext=(theoryData.loc[tangencyIdx,"sigma_P"],theoryData.loc[tangencyIdx,"value"]), # text position (x, y)

color='black',

fontsize=15)


plt.xlabel('Portfolio Risk')

plt.ylabel('Portfolio Expected Return')
```

```
plt.savefig("3stocks_tangency.pdf")



import matplotlib.pyplot as plt
import numpy as np

# ---------------------------
# PARAMETERS
# ---------------------------
risk_free_rate = 0.001 # 0.1% per month, as given
target_return = 0.01 # 1% per month target return

# ---------------------------
# SET UP THE FIGURE
# ---------------------------
plt.figure(figsize=(20, 10), dpi=80)

# ---------------------------
# PLOT STOCK-ONLY PORTFOLIOS (Efficient Frontier)
# ---------------------------
plt.plot(graphData_f['sigma_P'], graphData_f['value'],
color='red',
dash_capstyle='round',
linestyle=(0, (0.1, 2)),
linewidth=4,
```

```
marker='o',
markersize=6,
label="Efficient Frontier (Stocks Only)")

# ---------------------------
# PLOT THE CAPITAL MARKET LINE (CML)
# ---------------------------
plt.plot(graphData_c['sigma_P'], graphData_c['value'],
color='blue',
dash_capstyle='round',
linestyle=(0, (0.1, 2)),
linewidth=4,
marker='s',
markersize=6,
label="Capital Market Line (CML)")

# ---------------------------
# PLOT THE RISK-FREE ASSET
# ---------------------------
# The risk-free asset is represented by zero risk and its guaranteed return.
plt.scatter(0, risk_free_rate, color='green', s=150, zorder=5, label="Risk-Free Asset")

# ---------------------------
# PLOT THE TANGENCY PORTFOLIO
# ---------------------------
tan_x = theoryData.loc[tangencyIdx, "sigma_P"]
```

```
tan_y = theoryData.loc[tangencyIdx, "value"]
tan_y2 = S_port["mu_p"]

plt.scatter(tan_x, tan_y, color='black', s=150, zorder=5, label="Tangency Portfolio")
plt.annotate("T",
xy=(tan_x, tan_y),
xytext=(tan_x + 0.005, tan_y + 0.005),
textcoords="data",
arrowprops=dict(facecolor='black', arrowstyle="->"),
fontsize=20,
color='black')

# ---------------------------
# COMPUTE THE OPTIMAL MIX
# ---------------------------
# Compute the optimal weight (w*) to invest in the tangency portfolio in order to hit the target return:
optimal_weight = (target_return - risk_free_rate) / (tan_y2 - risk_free_rate)
# The risk (σ) of the resulting optimal (combined) portfolio is:
target_sigma = optimal_weight * tan_x

plt.scatter(target_sigma, target_return, color='purple', s=150, zorder=5,
label="Optimal Mix for 1% Target")
plt.annotate(f"Optimal Mix\nw = {optimal_weight:.2f}",
xy=(target_sigma, target_return),
xytext=(target_sigma + 0.005, target_return - 0.005),
textcoords="data",
```

```
arrowprops=dict(facecolor='purple', arrowstyle="->"),
fontsize=16,
color='purple')

# ---------------------------
# FIND THE STOCK-ONLY PORTFOLIO NEAR 1% RETURN
# ---------------------------
# Identify the portfolio on the efficient frontier that gets closest to the 1% expected return.
abs_diff = abs(graphData_f['value'] - target_return)
idx_stock_only = abs_diff.idxmin()
stock_only_sigma = graphData_f['sigma_P'].loc[idx_stock_only]
stock_only_return = graphData_f['value'].loc[idx_stock_only]

plt.scatter(stock_only_sigma, stock_only_return, color='orange', s=150, zorder=5,
label="Stock-Only Portfolio")
plt.annotate("Stock Only",
xy=(stock_only_sigma, stock_only_return),
xytext=(stock_only_sigma + 0.005, stock_only_return + 0.005),
textcoords="data",
arrowprops=dict(facecolor='orange', arrowstyle="->"),
fontsize=16,
color='orange')

# ---------------------------
# DRAW A COMPARISON LINE
# ---------------------------
```

```
# This line shows the risk reduction benefit from mixing the tangency portfolio with the risk-free asset
plt.plot([stock_only_sigma, target_sigma], [stock_only_return, target_return],
color='gray', linestyle='--', linewidth=2)
plt.text((stock_only_sigma + target_sigma)/2, (stock_only_return + target_return)/2,
"Risk reduction via mixing risk-free asset", fontsize=14,
color='gray', ha='center', va='bottom', rotation=30)

# ---------------------------
# FINALIZE THE PLOT
# ---------------------------
plt.xlabel('Portfolio Risk (σ)', fontsize=16)
plt.ylabel('Expected Return (E[R])', fontsize=16)
plt.title('Optimal Portfolio Selection and Risk-Free Mixing for a 1% Monthly Return Target', fontsize=20)
plt.grid(True, which='both', linestyle='--', linewidth=0.5)
plt.legend(fontsize=14)
plt.tight_layout()

# Save and display the figure
plt.savefig("3stocks_tangency_optimized.pdf", bbox_inches='tight')
plt.show()

#Min VAR portfolio
print(Omega)
scalingFactor = float(np.dot(np.dot(onesVec,np.linalg.inv(Omega)),onesVec.transpose()))**-1
w_M = scalingFactor * np.dot(np.linalg.inv(Omega),onesVec.transpose())
mu_M = np.dot(w_M.transpose(),mu)
```

```
sigma_M = float(np.dot(np.dot(w_M.transpose(),Omega),w_M))**(1/2)



#Multiple regression

# prompt: select the first row from the df_portfolios_sorted_M and put it inside a table all these values


S_port = df_portfolios_sorted_S.iloc[0]

print(S_port)

print("An investor who wants to maximise Sharpe ratio. Portfolio S:")

print(S_port["w1_O"], S_port["w2_LNG"], S_port["w3_MS"], S_port["w4_Cost"], S_port["w5_BYDE"],
S_port["w6_ASML"])

print(df_stocks)

print(df_stocks["date"])

#["O", "LNG", "MS", "COST", "BYD", "ASML"]

w=np.array([S_port["w1_O"],S_port["w2_LNG"],S_port["w3_MS"],S_port["w4_Cost"],S_port["w5_BYDE"],
S_port["w6_ASML"]])

print(w)


O = S_port["w1_O"] * df_stocks["O"]

LNG = S_port["w2_LNG"] * df_stocks["LNG"]

MS = S_port["w3_MS"] * df_stocks["MS"]

COST = S_port["w4_Cost"] * df_stocks["COST"]

BYDE = S_port["w5_BYDE"] * df_stocks["BYDE"]

ASML = S_port["w6_ASML"] * df_stocks["ASML"]

sum = O+LNG+MS+COST+BYDE+ASML

print(sum)
```

```
# Compute the returns on this portfolio with weights w for each month

# Assuming df_stocks contains monthly returns for the assets
portfolio_returns = df_stocks[["O", "LNG", "MS", "COST", "BYDE", "ASML"]].dot(w)
print(portfolio_returns)
# Rename the column to 'Portfolio Returns'
portfolio_returns = portfolio_returns.rename('Portfolio Returns')

# Concatenate the DataFrames
portfolio_returns = pd.concat([df_stocks["date"], portfolio_returns], axis=1)
print(portfolio_returns)

# prompt: # plot the graph

# Assuming portfolio_returns is already defined as in the previous code

plt.figure(figsize=(10, 6))
plt.plot(portfolio_returns['date'], portfolio_returns['Portfolio Returns'])
plt.xlabel('Date')
plt.ylabel('Portfolio Returns')
plt.title('Portfolio Returns Over Time')
plt.grid(True)
plt.xticks(rotation=45) # Rotate x-axis labels for better readability
plt.tight_layout() # Adjust layout to prevent labels from overlapping
plt.show()
```

```
# Statistical features of the return series
print(portfolio_returns['Portfolio Returns'].describe())

# read the data from google drive: FR2209_assessment_data_factors.xlsx

df_factors = pd.read_excel('/content/drive/MyDrive/FR2209_assessment_data_factors.xlsx')

df_factors.head()

# prompt: Plot the returns on the factors and describe any interesting features

# Assuming df_factors is already loaded as in the previous code

# Plotting the returns on the factors
plt.figure(figsize=(12, 8))

for column in df_factors.columns[1:]: # Iterate through factor columns (excluding 'date')
plt.plot(df_factors['Date'], df_factors[column], label=column)

plt.xlabel('Date')
plt.ylabel('Factor Returns')
plt.title('Factor Returns Over Time')
plt.legend()
plt.grid(True)
plt.xticks(rotation=45)
```

```
plt.tight_layout()
plt.show()

# Describe statistical features of each factor return
for column in df_factors.columns[1:]:
print(f"\nStatistical features for {column}:")
print(df_factors[column].describe())

print(df_factors)
print(portfolio_returns)

# prompt: concat to dataframes base on the date and drop duplicated columns

# Merge the two dataframes based on the date column
merged_df = pd.merge(portfolio_returns, df_factors, left_on='date', right_on='Date', how='inner')

# Drop duplicated date column
merged_df = merged_df.drop(columns=['Date'])

# Now you can proceed with the multivariate regression on the merged DataFrame.
merged_df

# run the multiple linear regression using stats model

import statsmodels.api as sm
```

```
# Define the independent variables (factors) and dependent variable (portfolio returns)
X = merged_df[['XSMKT', 'SMB', 'HML', "RMW", "CMA"]] # Independent variables
y = merged_df['Portfolio Returns'] # Dependent variable

# Add a constant to the independent variables (intercept term)
X = sm.add_constant(X)

# Fit the OLS model
model = sm.OLS(y, X).fit()

# Print the regression results
print(model.summary())

# Access specific statistics from the results
print("\nCoefficients:", model.params)
print("\nR-squared:", model.rsquared)
print("\nP-values:", model.pvalues)

from statsmodels.stats.stattools import durbin_watson
from statsmodels.stats.diagnostic import acorr_ljungbox
from statsmodels.compat import lzip

# Serial Correlation Test (Ljung-Box)
print("\nLjung-Box Test for Serial Correlation:")
print("\nLjung-Box Test:")
print(acorr_ljungbox(model.resid, lags=[10]))
```

```
from statsmodels.stats.diagnostic import het_breuschpagan, het_white

# Heteroskedasticity Test (Breusch-Pagan)
print("\nBreusch-Pagan Test for Heteroskedasticity:")
bp_test = het_breuschpagan(model.resid, model.model.exog)
labels = ['Lagrange multiplier statistic', 'p-value', 'f-value', 'f p-value']
print(dict(zip(labels, bp_test)))

white_test = het_white(model.resid, model.model.exog)
print("\nWhite Test:")
print(white_test)

from statsmodels.stats.diagnostic import het_arch

# 'model' is your original OLS regression
arch_test = het_arch(model.resid)

print("ARCH Test for Heteroskedasticity:")
print(f"Test statistic: {arch_test[0]:.4f}")
print(f"P-value: {arch_test[1]:.4f}")


from statsmodels.stats.diagnostic import acorr_breusch_godfrey

bg_test = acorr_breusch_godfrey(model, nlags=5)
```

```
print("Breusch-Godfrey Test:")
print(f"LM stat: {bg_test[0]:.4f}, p-value: {bg_test[1]:.4f}")

# prompt: reestimate the model but now removing the variables that have a p value more then 10 %

# Assuming 'model' is the OLS regression model from the previous code

# Get p-values for each variable
p_values = model.pvalues

# Identify variables with p-values greater than 0.10
variables_to_remove = p_values[p_values > 0.15].index.tolist()

# Remove the constant term if present
if 'const' in variables_to_remove:
variables_to_remove.remove('const')

# Re-estimate the model without the insignificant variables
if variables_to_remove:
X_new = X.drop(columns=variables_to_remove)
model_new = sm.OLS(y, X_new).fit()
print(model_new.summary())

# Perform diagnostic tests again with the new model
print("\nLjung-Box Test for Serial Correlation:")
print(acorr_ljungbox(model_new.resid, lags=[10]))
```

```
print("\nBreusch-Pagan Test for Heteroskedasticity:")
bp_test_new = het_breuschpagan(model_new.resid, model_new.model.exog)
labels = ['Lagrange multiplier statistic', 'p-value', 'f-value', 'f p-value']
print(dict(zip(labels, bp_test_new)))

white_test_new = het_white(model_new.resid, model_new.model.exog)
print("\nWhite Test:")
print(white_test_new)

arch_test_new = het_arch(model_new.resid)
print("\nARCH Test for Heteroskedasticity:")
print(f"Test statistic: {arch_test_new[0]:.4f}")
print(f"P-value: {arch_test_new[1]:.4f}")

bg_test_new = acorr_breusch_godfrey(model_new, nlags=5)
print("\nBreusch-Godfrey Test:")
print(f"LM stat: {bg_test_new[0]:.4f}, p-value: {bg_test_new[1]:.4f}")

else:
print("No variables removed. All variables have p-values <= 0.10.")
```